

The F2Z polynomial commitment scheme

Relations, reductions, and a normative prover–verifier specification

F2Z is a polynomial-commitment layer for proving the Lin-BitsDual relation. Given a public vector $\mathbf{v}$ and a value $\mu \in \mathbb{F}_q$, the relation asserts

$$\langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu, \quad \pi_2(\mathbf{h}) = \mathbf{M}\pi_2(\mathbf{f}), \quad O = \text{Enc}_C(\text{Pack}(\pi_2(\mathbf{f}))),$$

where $\mathbf{f}$ is the committed binary witness, $\mathbf{h}$ is the active witness used by the application, and $\mathbf{M}$ is a public developer-selected virtualization map. Here $\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$ denotes coordinatewise reduction modulo 2. The application PIOP supplies this linear claim to F2Z. Under the configured layout of $\mathbf{h}$, an admissible claim has a separable coefficient matrix $V[r, c] = a_r a'_c$; the public factors $(\mathbf{a}, \mathbf{a}')$ are part of the F2Z instance.

Drawing inspiration from binary-field techniques used in systems such as Flock and Binius64, F2Z combines the following techniques:

- **Binary packing.** The committed witness bits are packed into elements of the binary extension field $\mathbb{K}$ and encoded using the binary linear code C . This allows the commitment and opening protocols to operate on packed field elements.
- **Witness virtualization.** The public map $\mathbf{M}$ relates the committed witness $\mathbf{f}$ to the witness $\mathbf{h}$ on another ring/field. This permits constraints to be layered on top of each other while making the witness $\mathbf{f}$ to be small.
- **Chunked integer lifting.** Coefficients in $\mathbb{F}_q$ are lifted to canonical integers and split into bounded chunks. This preserves the original modular claim while keeping every intermediate fold within the injective range of the binary-field exponent encoding.
- **Grand-product encoding.** Because the witness coordinates are binary, each bounded linear fold can be expressed as a product identity over $\mathbb{K}$. This converts the original linear claim into product claims suited to binary-field verification.
- **GKR and LinCheck reductions.** GKR verifies the product computation succinctly, while the row LinCheck reduces the resulting weighted claim to a multilinear-extension evaluation of the active witness.
- **Adjoint pullback.** The verifier transports the final coefficient through $\mathbf{M}^\top$, converting a claim on $\mathbf{h}$ into a claim on the originally committed witness $\mathbf{f}$ without another commitment or protocol message.
- **Conditional reduction, ring switching, and batching.** The coefficient LinCheck is used only when the pulled-back coefficient is not already in the required MLE form. A ring switch then converts the resulting evaluations into packed linear claims, which are batched into a single recursive Ligerito opening.

Together, these techniques allow F2Z to prove inner products claims over $\mathbb{F}_q$, and therefore support constraints beyond $\mathbb{F}_2$, while retaining a binary-field commitment. This specification fixes the Lin-BitsDual relation, its prover–verifier reductions, and its concrete transcript and encoding rules.

1 Overview and interfaces

1.1 Committed and active witness views

The PCS receives a committed binary witness $\mathbf{f}$. Before proving, the developer selects one of two public witness configurations:

- **Direct.** The application PIOP uses the committed witness directly, so $\mathbf{h} = \mathbf{f}$ and $\mathbf{M} = I$.
- **Virtualized.** The application PIOP uses a binary witness $\mathbf{h}$ related to the committed witness by a nontrivial developer-selected public matrix $\mathbf{M}$.

The remainder of this specification uses the developer’s Virtualized configuration exclusively. Throughout, $\mathbf{f}$ is the commitment-facing view and $\mathbf{h}$ is the application-facing view.

Symbol	Type	Role
$\mathbf{f}$	$\{0, 1\}^{n_f}$	Witness committed by the PCS.
$\mathbf{M}$	$\mathbb{F}_2^{n_h \times n_f}$	Developer-selected virtualization matrix.
$\mathbf{h}$	$\{0, 1\}^{n_h}$	Active witness used by the PIOP and the F2Z core.
O	$\mathbb{K}^{n_O}$	Codeword oracle encoding the packed coordinates of $\mathbf{f}$.
ρ	bit string	Merkle root of O .

The active witness is

$$\pi_2(\mathbf{h}) = \mathbf{M}\pi_2(\mathbf{f}). \quad (1)$$

Here $\pi_2 : \mathbb{Z} \rightarrow \mathbb{F}_2$, $x \mapsto x \bmod 2$, acts coordinatewise. Since $\mathbf{f}$ and $\mathbf{h}$ are binary, Equation (1) uniquely determines $\mathbf{h}$. The developer fixes the public map $\mathbf{M}$ before proving; it is part of the instance and is not chosen by the prover.

Only $\mathbf{f}$ is committed. Let F be its fixed two-dimensional layout and let $\text{vec}(F) = \pi_2(\mathbf{f})$. The commitment is

$$P := \text{Pack}(F), \quad O := \text{Enc}_C(P), \quad \rho := \text{Root}(O). \quad (2)$$

Neither H , the layout of $\mathbf{h}$, nor any value derived from H is encoded in O .

1.2 Application linear-claim handoff

The application begins with a constraint-satisfaction relation over $\mathbb{F}_q$ evaluated on the active witness $\pi_q(\mathbf{h})$. Its PIOP reduces that relation to an admissible Lin-BitsDual claim consumed by F2Z:

$$\begin{aligned} (x; \pi_q(\mathbf{h})) \in \mathcal{R}_{\text{app}} &\xrightarrow{\Pi_{\text{app}}} \langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu, \quad V = \mathbf{a}(\mathbf{a}')^\top \\ &\xrightarrow{\text{F2Z}} \text{opening against } \rho. \end{aligned} \quad (3)$$

The application proof is outside the F2Z protocol. F2Z receives the public claim and does not inspect the equations or reductions that produced it. The remainder of this specification defines the public parameters and reduces this linear claim to an opening against the commitment to $\mathbf{f}$.

2 Instance and parameters

2.1 Domains and representations

Object	Concrete value	Rule
$\mathbb{F}_2$	binary field	Matrix $\mathbf{M}$ and witness coordinates
$R = \mathbb{F}_q$	$q = 2^{100} - 15$	Application and F2Z scalar linear claims
$\mathbb{K}$	$\mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$	Packing, products, GKR, and openings
π_2	$\mathbb{Z} \rightarrow \mathbb{F}_2$	Coordinatewise reduction modulo 2
π_q	$\mathbb{Z} \rightarrow \mathbb{F}_q$	Reduction modulo q
$\gamma(a)$	$\gamma(a) \in [0, q), \pi_q(\gamma(a)) = a$	Unique representative; no alternate lift is accepted
β_v	$X^v, 0 \leq v < 128$	Ordered $\mathbb{F}_2$ -basis of $\mathbb{K}$

$$R = \mathbb{F}_q, \quad q = 2^{100} - 15, \quad \mathbb{K} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1). \quad (4)$$

2.2 Witnesses and layouts

View	Vector	Layout	Used by
Committed	$\mathbf{f} \in \{0, 1\}^{n_f}$	$F \in \mathbb{F}_2^{2^{m_f-7} \times 2^7}$	packing, coding, and opening
Active	$\mathbf{h} \in \{0, 1\}^{n_h}$	$H \in \mathbb{F}_2^{2^{d_r} \times 2^{d_c}}$	PIOP, folds, products, and GKR

The layouts satisfy

$$\text{vec}(F) = \pi_2(\mathbf{f}), \quad \text{vec}(H) = \pi_2(\mathbf{h}), \quad \text{vec}(H) = \mathbf{M} \text{vec}(F). \quad (5)$$

Packing uses only F . For $0 \leq j < n_f/128$,

$$P[j] := \sum_{v=0}^{127} F_{128j+v} \beta_v, \quad O := \text{Enc}_{\mathcal{C}}(P), \quad \rho := \text{Root}(O), \quad (6)$$

where F_i denotes the i -th coordinate in the fixed vectorization. Thus $P \in \mathbb{K}^{n_f/128}$ before code padding. The public instance requires n_f to be a multiple of 128 after witness padding.

2.3 Separable claim and bounded fold decomposition

F2Z receives the public linear claim

$$\langle \mathbf{v}, \pi_q(\mathbf{h}) \rangle_{\mathbb{F}_q} = \mu. \quad (7)$$

The developer configuration fixes $H \in \mathbb{F}_2^{2^{d_r} \times 2^{d_c}}$ and its row-major vectorization. Using the same coordinate order, reshape $\mathbf{v}$ into $V \in \mathbb{F}_q^{2^{d_r} \times 2^{d_c}}$ by

$$V[r, c] := v_{r \cdot 2^{d_c} + c}. \quad (8)$$

Identifying each bit $H[r, c]$ with its canonical image in $\mathbb{F}_q$, Equation (7) is

$$\sum_{r,c} V[r, c] H[r, c] = \mu. \quad (9)$$

The F2Z fold accepts only a separable coefficient matrix. An admissible instance therefore includes public factors $\mathbf{a} = (a_r)_r$ and $\mathbf{a}' = (a'_c)_c$ satisfying

$$V[r, c] = a_r a'_c, \quad \mu = \sum_c a'_c \sum_r a_r H[r, c]. \quad (10)$$

An application that begins with a nonseparable coefficient must perform its own reduction before invoking F2Z.

For every row factor $a_r \in \mathbb{F}_q$, let $\gamma_r := \gamma(a_r) \in [0, q)$ be its canonical integer representative. Since $q < 2^{100}$, each γ_r has at most 100 bits. For $d_r \leq 126$, define

$$c_w := 127 - d_r, \quad L := \left\lceil \frac{100}{c_w} \right\rceil. \quad (11)$$

Split each lift into L little-endian chunks:

$$\gamma_r^{(\ell)} := \lfloor \gamma_r / 2^{c_w \ell} \rfloor \bmod 2^{c_w}, \quad \gamma_r = \sum_{\ell=0}^{L-1} 2^{c_w \ell} \gamma_r^{(\ell)}. \quad (12)$$

For every column c and chunk ℓ , define the exact integer fold

$$\mu_c^{(\ell)} := \sum_{r < 2^{d_r}} \gamma_r^{(\ell)} H[r, c] \in \mathbb{Z}. \quad (13)$$

Because $0 \leq \gamma_r^{(\ell)} < 2^{c_w}$ and $H[r, c] \in \{0, 1\}$,

$$0 \leq \mu_c^{(\ell)} < 2^{c_w + d_r} = 2^{127}. \quad (14)$$

The verifier reconstructs the supplied field claim by reducing the exact folds modulo q , applying the column factors, and summing the chunks:

$$\mu = \sum_c a'_c \sum_{\ell=0}^{L-1} \pi_q(2^{c_w \ell} \mu_c^{(\ell)}). \quad (15)$$

The product generator has full multiplicative order:

$$g \in \mathbb{K}^\times, \quad \text{ord}(g) = 2^{128} - 1. \quad (16)$$

Together, Equations (14) and (16) make $u \mapsto g^u$ injective on every accepted fold. Section 3 uses this injectivity to convert each fold into a grand-product claim over $\mathbb{K}$.

2.4 Protocol interfaces

Protocol	Input	Output
Π_1	separable Lin-BitsDual claim and chunk schedule	exact folds and grand-product claims on H
Π_{GKR}	product claims and the fixed product circuit	weighted sum claim on H
Π_{MLE}	weighted sum claim on H	$\tilde{H}(\mathbf{r}_h) = \eta$
$\text{Adj}_{\mathbf{M}}$	$\mathbf{a}_h$	$\mathbf{a}_f = \mathbf{M}_{\mathbb{K}}^T \mathbf{a}_h$; no message
$\Pi_{\text{LC}}(\text{LinCheck}_f)$	$\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle = \eta$	MLE-shaped claim when required
Π_{iopp}	MLE-shaped claim and O	accepted opening or rejection

The coefficient protocol LinCheck_f is omitted when $\mathbf{a}_f$ already has the MLE shape required by the opening protocol. Omission depends only on its shape in the committed packing layout.

3 Relations, reductions, and proof objects

An encoded relation contains triples $(x, O; \text{wit})$. The explicit instance is x , the encoded oracle is O , and wit is the witness view used by that relation. Every relation below carries the same public instance parameters, matrix $\mathbf{M}$, oracle O , and root $\rho = \text{Root}(O)$. The binding invariant is always

$$O = \text{Enc}_C(\text{Pack}(F)), \quad \text{vec}(F) = \pi_2(\mathbf{f}). \quad (17)$$

No intermediate relation replaces F by H in Equation (17).

The edge notation distinguishes deterministic claim changes from interactive reductions:

$$\mathcal{R}_0 \equiv_G \mathcal{R}_1 \quad \text{and} \quad \mathcal{R}_0 \xRightarrow{\Pi} \mathcal{R}_1. \quad (18)$$

The first edge applies a public deterministic map G and sends no message. The second edge is implemented by the named protocol.

3.1 Relation chain

F2Z begins from the application-supplied separable Lin-BitsDual claim:

$$\begin{aligned} \mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}] := \left\{ ((\mathbf{a}, \mathbf{a}', \mu, \mathbf{M}), O; \mathbf{f}) : \text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \right. \\ \mu = \sum_c a'_c \sum_r a_r H[r, c], \\ \left. O = \text{Enc}_C(\text{Pack}(F)), \quad \text{vec}(F) = \pi_2(\mathbf{f}) \right\}. \end{aligned} \quad (19)$$

The F2Z statement binds $(\mathbf{a}, \mathbf{a}', \mu)$, the active layout H , the chunk schedule, and the virtualization matrix to the transcript. The application proof establishing this claim is outside F2Z, and no second commitment is created.

For each active chunk ℓ , define

$$Q_c^{(\ell)}(H) := \prod_r \left(1 + (g^{\gamma_r^{(\ell)}} - 1)H[r, c] \right) - g^{\mu_c^{(\ell)}}. \quad (20)$$

The product relation is

$$\begin{aligned} \mathcal{R}_{\text{PESAT}}[\mathbf{h}] := \left\{ ((\gamma^{(\ell)}, \mu^{(\ell)}, \mathbf{M}), O; \mathbf{f}) : \text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \quad Q_c^{(\ell)}(H) = 0 \ (\forall c, \ell), \right. \\ \left. O = \text{Enc}_C(\text{Pack}(F)), \quad \text{vec}(F) = \pi_2(\mathbf{f}) \right\}. \end{aligned} \quad (21)$$

The exact folds $\mu_c^{(\ell)}$ are checked against the supplied linear claim by Equation (15) before this relation is accepted.

GKR reduces the product computation to a weighted sum claim. Its verifier derives public weights ω_ξ and a target σ :

$$\mathcal{R}_{\text{sum}}^h := \left\{ ((\omega_\xi, \sigma, \mathbf{M}), O; \mathbf{f}) : \sum_z \omega_{\xi, z} H[z] = \sigma, \quad \text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \quad \text{Equation (17)} \right\}. \quad (22)$$

GKR reads the active witness through its circuit oracle. It does not query O or assert that O encodes H .

The row protocol LinCheck_H reduces the weighted sum claim to one MLE evaluation:

$$\mathcal{R}_{\text{eval}}^h := \left\{ ((\mathbf{r}_h, \eta, \mathbf{M}), O; \mathbf{f}) : \tilde{H}(\mathbf{r}_h) = \eta, \quad \text{vec}(H) = \mathbf{M}\pi_2(\mathbf{f}), \quad \text{Equation (17)} \right\}. \quad (23)$$

For $z \in \{0, 1\}^{m_h}$ and $\mathbf{r}_h \in \mathbb{K}^{m_h}$, define

$$\text{eq}(z, \mathbf{r}_h) := \prod_{j=0}^{m_h-1} \left((1 - z_j)(1 - r_{h,j}) + z_j r_{h,j} \right). \quad (24)$$

By the definition of the multilinear extension,

$$\tilde{H}(\mathbf{r}_h) = \sum_{z \in \{0, 1\}^{m_h}} \text{eq}(z, \mathbf{r}_h) H[z]. \quad (25)$$

Thus the MLE claim has coefficient vector $\mathbf{a}_h[z] := \text{eq}(z, \mathbf{r}_h)$. Scalar extension and the adjoint give

$$\mathbf{a}_f := \mathbf{M}_{\mathbb{K}}^T \mathbf{a}_h, \quad \langle \mathbf{a}_h, \mathbf{h}_{\mathbb{K}} \rangle_{\mathbb{K}} = \langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta. \quad (26)$$

This step sends no message. The public instance must provide a deterministic full-adjoint evaluator, for example through

$$\widetilde{\mathbf{M}_{\mathbb{K}}^T \mathbf{a}_h}(\mathbf{r}_f) = \left\langle \mathbf{a}_h, \mathbf{M}_{\mathbb{K}}(\text{eq}(\cdot, \mathbf{r}_f)) \right\rangle_{\mathbb{K}}. \quad (27)$$

The resulting encoded relation is

$$\begin{aligned} \mathcal{R}_{\text{Lin}}^f := \left\{ ((\mathbf{a}_f, \eta), O; \mathbf{f}) : \mathbf{a}_f \in \mathbb{K}^{n_f}, \quad \langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta, \right. \\ \left. O = \text{Enc}_C(\text{Pack}(F)), \quad \text{vec}(F) = \pi_2(\mathbf{f}) \right\}. \end{aligned} \quad (28)$$

Relation	Public claim	Active witness	Commitment invariant	Successor
$\mathcal{R}_{\text{LinBitsDual}}[\mathbf{h}]$	$\sum_c a'_c \sum_r a_r H[r, c] = \mu$	$\mathbf{h}$	O encodes $\mathbf{f}$	Π_1
$\mathcal{R}_{\text{PESAT}}[\mathbf{h}]$	product identities	H	unchanged	Π_{GKR}
$\mathcal{R}_{\text{sum}}^h$	weighted sum on H	H	unchanged	Π_{MLE}
$\mathcal{R}_{\text{eval}}^h$	$\tilde{H}(\mathbf{r}_h) = \eta$	H	unchanged	$\mathbf{M}^T$
$\mathcal{R}_{\text{Lin}}^f$	$\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle = \eta$	$\mathbf{f}$	O encodes $\mathbf{f}$	Π_{iopp}
$\mathcal{R}_{\text{triv}}$	acceptance	—	discharged	—

3.2 Reductions

The complete relation graph is

$$\begin{aligned} \mathcal{R}_{\text{LinBitsDual}}[h] &\xrightarrow{\Pi_1} \mathcal{R}_{\text{PESAT}}[h] \xrightarrow{\Pi_{\text{GKR}}} \mathcal{R}_{\text{sum}}^h \\ &\xrightarrow{\Pi_{\text{MLE}}} \mathcal{R}_{\text{eval}}^h \equiv_{M^T} \mathcal{R}_{\text{Lin}}^f \xrightarrow{\Pi_{\text{iopp}}} \mathcal{R}_{\text{triv}}. \end{aligned} \quad (29)$$

Integer folds and products. Π_1 sends the active exact folds

$$\pi_1 := \left(\nu^{(\ell)}, \mu^{(\ell)} \right)_{\ell \in \mathcal{A}^\uparrow}, \quad \nu_c^{(\ell)} := g^{\mu_c^{(\ell)}}. \quad (30)$$

The verifier requires $0 \leq \mu_c^{(\ell)} < 2^{127}$, checks $g^{\mu_c^{(\ell)}} = \nu_c^{(\ell)}$, and checks Equation (15) before deriving the product instances. The vectors $\nu^{(\ell)}$ are proof records. The integer vectors $\mu^{(\ell)}$ are checked, non-absorbed hints. Inactive chunks have derived value zero.

GKR and row LinCheck. For every active chunk, the fixed GKR interface consumes one product instance and returns one weighted sum claim on H :

$$(\text{tr}', \omega_\xi, \sigma) \leftarrow \text{GKR.Verify}(\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}, \pi_{\text{GKR}}; \text{tr}). \quad (31)$$

The row LinCheck then consumes that sum claim and returns $(\mathbf{r}_h, \eta)$ for Equation (23). Write its proof fragment as π_{MLE} . The existing GKR wire record carries the ordered pair $(\pi_{\text{GKR}}, \pi_{\text{MLE}})$; this semantic split does not add a wire tag or change framing.

Adjoint and opening. The verifier evaluates $\mathbf{a}_f = M_{\mathbb{K}}^T \mathbf{a}_h$ locally. If $\mathbf{a}_f$ is already MLE-shaped, Π_{iopp} proceeds directly to the ring switch. Otherwise it first runs the coefficient protocol LinCheck_f. The decision is determined by the public instance and coefficient shape, not by a prover flag.

3.3 Opening a pulled-back claim

For a general coefficient, the coefficient protocol LinCheck_f proves

$$\sum_{x \in \{0,1\}^{m_f}} \tilde{a}_f(x) \tilde{f}_{\mathbb{K}}(x) = \eta. \quad (32)$$

Starting with $\sigma_{-1} := \eta$, round i sends a degree-two polynomial g_i and checks

$$g_i(0) + g_i(1) = \sigma_{i-1}, \quad r_i \leftarrow \mathbb{K}, \quad \sigma_i := g_i(r_i), \quad 0 \leq i < m_f. \quad (33)$$

Let $\mathbf{r} = (\mathbf{r}_{\text{lo}}, \mathbf{r}_{\text{hi}})$, with $|\mathbf{r}_{\text{lo}}| = 7$. The first ring-switch message derives

$$z := \sum_{v=0}^{127} \text{eq}(v, \mathbf{r}_{\text{lo}}) s_v = \tilde{f}_{\mathbb{K}}(\mathbf{r}), \quad \sigma_{m_f-1} = \tilde{a}_f(\mathbf{r}) z. \quad (34)$$

The verifier performs this check after absorbing the message and before its next squeeze. It uses no division and has no exceptional branch when $\tilde{a}_f(\mathbf{r}) = 0$.

The coefficient-LinCheck fragment is $\pi_{\text{LC}} := (g_i)_{i=0}^{m_f-1}$ and has error at most $\varepsilon_{\text{LC}} := 2m_f/|\mathbb{K}|$. An MLE-shaped coefficient sets $\pi_{\text{LC}} = ()$ and $\varepsilon_{\text{LC}} = 0$. For $0 \leq v < 128$, let F_v be the v -th bit plane of the packed committed layout. The prover transmits

$$\mathbf{s} = (s_v)_{v < 128}, \quad s_v := \tilde{F}_v(\mathbf{r}_{\text{hi}}). \quad (35)$$

The verifier derives a packed claim

$$O = \text{Enc}_C(P), \quad \langle B, P \rangle_{\mathbb{K}} = \beta. \quad (36)$$

For several active chunks, fresh coefficients η_{bat} combine the packed claims without adding a proof field:

$$B := \sum_{\ell \in \mathcal{A}} \eta_{\text{bat}, \ell} B_\ell, \quad \beta := \sum_{\ell \in \mathcal{A}} \eta_{\text{bat}, \ell} \beta_\ell. \quad (37)$$

The recursive constrained-code proof π_{lig} verifies this claim against ρ .

3.4 Proof object and soundness

The semantic proof object is

$$\pi_{\text{F2Z}} = \left(\left(\nu^{(\ell)}, \mu^{(\ell)}, \pi_{\text{GKR}}^{(\ell)}, \pi_{\text{MLE}}^{(\ell)} \right)_{\ell \in \mathcal{A}^\uparrow}, \pi_{\text{LC}}, (\mathbf{s}^{(\ell)})_{\ell \in \mathcal{A}^\uparrow}, \pi_{\text{lig}} \right). \quad (38)$$

Fragment	Successor claim	Error
$\pi_1^{(\ell)}$	product claims on H	ε_1
$\pi_{\text{GKR}}^{(\ell)}$	weighted sum claim on H	$\varepsilon_{\text{GKR}}^{(\ell)}$
$\pi_{\text{MLE}}^{(\ell)}$	MLE claim on H	$\varepsilon_{\text{MLE}}^{(\ell)}$
π_{LC}	MLE-shaped claim on F	ε_{LC} or 0
ring switch and batch	packed claim	$\varepsilon_{\text{ring}}^{(\ell)}, \varepsilon_{\text{batch}}$
π_{lig}	acceptance	$\varepsilon_{\text{open}}$

The v1 wire format partitions transmitted fields, in replay order, as

$$\begin{aligned} \pi_{\text{wire}} &:= (N_{\text{F2Z}}, H_{\text{F2Z}}), & \text{narglen}(\pi) &:= |N_{\text{F2Z}}|, & \text{hintlen}(\pi) &:= |H_{\text{F2Z}}|, \\ & & \text{payloadlen}(\pi) &:= |N_{\text{F2Z}}| + |H_{\text{F2Z}}|, & \text{wirelen}(\pi) &:= |\text{HostEncode}_{v1}(N_{\text{F2Z}}, H_{\text{F2Z}})|. \end{aligned} \quad (39)$$

The byte tags and container rules are fixed in Section 6.4.

The composed error is bounded by

$$\begin{aligned} \varepsilon_{\text{F2Z}} &\leq \varepsilon_1 + \sum_{\ell \in \mathcal{A}} \left(\varepsilon_{\text{GKR}}^{(\ell)} + \varepsilon_{\text{MLE}}^{(\ell)} + \varepsilon_{\text{ring}}^{(\ell)} \right) \\ &\quad + \varepsilon_{\text{LC}} + \varepsilon_{\text{batch}} + \varepsilon_{\text{open}}. \end{aligned} \quad (40)$$

4 Prover–verifier interaction

4.1 Transcript channels and schedule

Equation (39) defines the two proof streams; these badges remain normative without color.

Badge	Bytes	Verifier action
$N[x]$	NARG	Consume a canonical typed record and absorb that same encoding before any dependent squeeze; PoW alone uses the checked-deferred read in Section 6.6.
$H[x]$	Hint	Consume, never absorb, and check against the bound prefix before the next squeeze.
$D[x]$	None	Public, verifier-derived, or challenge value; no proof bytes, with the exact framing and squeeze transition of Section 6.6.
$N/H[\pi]$	Both	The subordinate ledger fixes each field's channel and both replay orders.

Object notation $\text{Verify}(\dots, \pi; \text{tr})$ denotes the equivalent $\text{VerifyStream}(\dots; \text{tr}, c_N, c_H)$. It follows the subordinate ledger over the fixed proof system's bounded input slices, performs the same checks and events, and succeeds only at both declared cursor endpoints. Given public data and bound NARG prefix A , a hint h is admissible only if its accepted value is computationally unique:

$$\Pr[\exists h \neq h' : \text{CheckHint}_i(A, h) = \text{CheckHint}_i(A, h') = 1] \leq \text{negl}(\lambda). \quad (41)$$

The uniqueness condition in Equation (41) MUST be checked before the next **Squeeze**. Acceptance requires no pending hint and exact exhaustion of both cursors; missing, misordered, or trailing records reject.

Let $\text{Frame}_{V1}(\tau, \ell, k, i; z)$ be the byte frame of Equation (57). Before the first proof challenge,

$$\text{tr}_0 := \text{Absorb}(\text{Absorb}(\text{tr}_{\text{init}}, \text{Frame}_{V1}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{DomainV1})), \text{Frame}_{V1}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1})). \quad (42)$$

The payloads in Equation (42) are fixed by Equation (59). Later absorbs are domain-separated by their numeric tag and fixed (ℓ, k, i) scope fields. A squeeze updates tr and yields a derived, non-transmitted value; each populated interaction row denotes one event.

The tables realize Equation (29) as matched prover/verifier steps in this exact order:

$$\begin{aligned} & (P_1/V_1); \\ & \text{for } \ell \in \mathcal{A} \text{ in increasing order : } ((P_2/V_2)^{(\ell)}; (P_3/V_3)^{(\ell)}; \\ & (P_4/V_4); (P_5/V_5); (P_6/V_6); (P_7/V_7); (P_8/V_8); (P_9/V_9)). \end{aligned} \quad (43)$$

Equation (43) is the normative top-level replay order. Right ledgers contain post-step claims; *internal* claims and superscript- (ℓ) copies add no paper relation.

P1/V1. Bind the F2Z statement and commitment

Prover P	Traffic	Verifier V
Construct $\pi_2(h) = M\pi_2(f), \quad h \in \{0, 1\}^{n_h},$ $P := \text{Pack}(F), \quad \text{vec}(F) = \pi_2(f),$ $O := \text{Enc}_C(P), \quad \rho := \text{Root}(O).$	$\xrightarrow{D[\text{com}]}$	Validate the public factors (a, a') , claim value μ , configuration, shapes, code, and root type; then set $\text{tr} \leftarrow \text{Absorb}_{\text{statement}}(\text{tr}, (\text{pp}, a, a', \mu, M, \text{com}))$. Derive the unique integer representatives, chunk schedule, and active-chunk set from the public factors. This step sends no message and samples no challenge. Encode CoreSeamV1 by Equation (60) and absorb its tag-0003 frame at scope $(\perp_{32}, \perp_{32}, \perp_{32})$ before the first chunk record.
Retain h and its active matrix form H .		

REDUCTION LEDGER

CURRENT STATE

$$\text{LIN}[h] \text{ RELATION NEW} \\ \sum_c a'_c \sum_r a_r H[r, c] = \mu$$

WHERE

$$\rho \quad \rho = \text{Root}(O)$$

P2/V2. Run chunk ℓ of Π_1

Prover P	Traffic	Verifier V
Before sending anything, compute $\mu_c^{(\ell)} = \sum_r \gamma_r^{(\ell)} H[r, c], \quad \nu_c^{(\ell)} = g^{\mu_c^{(\ell)}}.$	$\xrightarrow{N[\nu^{(\ell)}]}$ $\xrightarrow{H[\mu^{(\ell)}]}$ $\xleftarrow{D[\zeta^{(\ell)}]}$	Read and absorb the canonical typed vector $\nu^{(\ell)}$. Before any later squeeze, require $ \mu^{(\ell)} = 2^{d_c}, \quad 0 \leq \mu_c^{(\ell)} < 2^{127},$ and $g^{\mu_c^{(\ell)}} = \nu_c^{(\ell)}$ for every c . The order of g and the range bound make this accepted exponent unique. $\zeta^{(\ell)} \leftarrow \text{Squeeze}_{\mathbb{K}}^s(\text{tr}),$ Then set $e_0^{(\ell)} := \widetilde{\nu^{(\ell)}}(\zeta^{(\ell)}).$

REDUCTION LEDGER

$$\text{PAPER-EDGE FRAGMENT } \Pi_1^{(\ell)} \\ \text{FROM } \mathcal{R}_{\text{LinBitsDual}}[h] \\ \text{TO } \mathcal{R}_{\text{PESAT}}^{(\ell)}[h]$$

CURRENT STATE

$$\text{LIN}[h] \text{ RELATION USED} \\ \sum_c a'_c \sum_r a_r H[r, c] = \mu$$

$$\text{PESAT}^{(\ell)} \text{ RELATION NEW} \\ Q_c^{(\ell)}(H) = 0 \quad (\forall c)$$

WHERE

$$\rho \quad \rho = \text{Root}(O)$$

$$P \quad P = \text{Pack}(F)$$

P3/V3. Run the GKR reduction for chunk ℓ

The PCS-level schedule treats GKR and the following row LinCheck as two reductions inside the existing GKR record. GKR consumes the current PESAT claim and returns a weighted sum claim on H . The row LinCheck consumes that claim and returns $\tilde{H}(r_h^{(\ell)}) = \eta_\ell$. Denote this typed input bundle by $\mathcal{T}_{\text{GKR}}^{(\ell)}$. Neither reduction queries O .

Prover P	Traffic	Verifier V
Construct the fixed GKR protocol's canonical proof fragment $\pi_{\text{red}}^{(\ell)} \leftarrow (\text{GKR}; \text{LinCheck}_H). \text{Prove}(\text{pp}_{\text{GKR}}, \mathcal{T}_{\text{GKR}}^{(\ell)}, H; \text{tr}).$	$\xrightarrow{N/H[\pi_{\text{red}}^{(\ell)}]}$	Compute $(\text{tr}', r_h^{(\ell)}, \eta_\ell) \leftarrow (\text{GKR}; \text{LinCheck}_H). \text{Verify}(\text{pp}_{\text{GKR}}, \mathcal{T}_{\text{GKR}}^{(\ell)}, \pi_{\text{red}}^{(\ell)}; \text{tr}),$ require success and complete consumption of the GKR fragment, then set $\text{tr} \leftarrow \text{tr}'.$

Retain the witness value $\tilde{H}(r_h^{(\ell)})$.

All internal GKR messages and challenges are owned by the fixed GKR protocol and use its domain namespace. Treating GKR atomically here does not permit an implementation to omit that subordinate transcript and encoding specification.

REDUCTION LEDGER

$$\text{ROW LINCHECK } \Pi_{\text{MLE}}^{(\ell)} \\ \text{FROM } \mathcal{R}_{\text{sum}}^{h, (\ell)} \\ \text{TO } \mathcal{R}_{\text{eval}}^{h, (\ell)}$$

CURRENT STATE

$$\text{LIN}[h] \text{ RELATION} \\ \sum_c a'_c \sum_r a_r H[r, c] = \mu$$

$$\text{EVAL}[H] \text{ RELATION NEW} \\ \tilde{H}(r_h^{(r)}) = \eta_r \\ (r \in \mathcal{A}, \quad r \leq \ell)$$

WHERE

$$\rho \quad \rho = \text{Root}(O)$$

$$P \quad P = \text{Pack}(F)$$

For the merged-forest GKR protocol, the repeated phase- x value $\mathbf{H}[S_{x,k}^{(\ell)}]$ is a valid hint because the prefix already fixes $e_k^{(\ell)}$ and the verifier immediately requires $S_{x,k}^{(\ell)} = e_k^{(\ell)}$ before any following squeeze. A canonical implementation SHOULD omit that redundant field and mark it $\mathbf{D}[e_k^{(\ell)}]$ instead. The phase- c claimed sums, sumcheck messages, and terminal child pairs remain $\mathbf{N}[\text{GKR messages}]$: their consistency checks do not uniquely determine them and later challenges must bind them. The outer encoded field `presums[ $\ell$ ].claimed_sums` is distinct from those phase- c claims. Wire format v1 omits it and supplies the derived value $\mathbf{D}[[e_d^{(\ell)} + 1]]$ to the MultiDegree row LinCheck. A decoder that encounters the reserved legacy tag 3081 MUST reject.

P4/V4. Complete the fold reconstruction check

Prover P	Traffic	Verifier V
		After all active chunks, set inactive $\mu^{(\ell)} := 0$, reconstruct $\mu_c^{\text{col}} := \sum_{\ell=0}^{L-1} 2^{cw\ell} \mu_c^{(\ell)} \in \mathbb{Z}$, and require the fold reconstruction equality $\sum_c a'_c \pi_q(\mu_c^{\text{col}}) = \mu$. This check has no message, absorb, or challenge.

REDUCTION LEDGER

PAPER COMPOSITION $\Pi_1; \Pi_{\text{GKR}}; \Pi_{\text{MLE}}$
FROM $\mathcal{R}_{\text{LinBitsDual}}[h]$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{eval}}^{h,(\ell)}$

CURRENT STATE

EVAL[H] **RELATION** **REDUCED**
 $\tilde{H}(r_h^{(\ell)}) = \eta_\ell$
 $(\ell \in \mathcal{A})$

WHERE

$\rho \ \rho = \text{Root}(O)$

P5/V5. Pull each active claim back through the adjoint

For each active claim, set

$$\mathbf{a}_h[z] := \text{eq}(z, \mathbf{r}_h), \quad \mathbf{a}_f := \mathbf{M}_{\mathbb{K}}^T \mathbf{a}_h, \quad \langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta. \quad (44)$$

Equation (44) generates no message or challenge. The verifier uses the developer-supplied full-adjoint evaluator of Equation (27).

REDUCTION LEDGER

ZERO-MESSAGE REWRITE $\text{Adj}_{\mathcal{M}_{\mathbb{K}}^T}$
FROM $\mathcal{R}_{\text{eval}}^h$
TO $\mathcal{R}_{\text{Lin}}^f$

CURRENT STATE

LIN[f] **RELATION** **NEW**
 $\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta$

WHERE

$\rho \ \rho = \text{Root}(O)$

P6/V6. Coefficient LinCheck when needed

P6/V6 is skipped when $\mathbf{a}_f$ already has the MLE shape required by the ring switch. Otherwise it runs the degree-two coefficient protocol LinCheck_f on the claim produced by P5/V5. Write $g_i(X) = \gamma_{i,0} + \gamma_{i,1}X + \gamma_{i,2}X^2$. For $i = 0, \dots, m_f - 1$, P6 sends its three canonical coefficients under NARG tag 4002 and scope $(\perp_{32}, \perp_{32}, i)$. V6 absorbs that frame, requires

$$g_i(0) + g_i(1) = \sigma_{i-1}, \quad \sigma_{-1} = \eta, \quad (45)$$

then squeezes $r_i \in \mathbb{K}$ under tag 4103 at the same scope and sets $\sigma_i = g_i(r_i)$. Equation (45) is checked before every dependent squeeze. Put $r = (r_{\text{lo}}, r_{\text{hi}})$, $|r_{\text{lo}}| = 7$. P6/V6 leaves the terminal gate

$$\sigma_{m_f-1} = \tilde{a}_f(r) \tilde{f}_{\mathbb{K}}(r), \quad (46)$$

Equation (46) is checked by P7/V7 with its first ring vector. No other coefficient LinCheck is permitted.

Prover P	Traffic	Verifier V
Send each degree-two g_i in increasing i .	$\xrightarrow{\mathbf{N}[\pi_{\text{LC}}]}$	Check, absorb, and squeeze each round in exact alternation; derive (r, σ_{m_f-1}) and the public evaluator $r \mapsto \tilde{a}_f(r)$.

REDUCTION LEDGER

$\Pi_{\text{IOPP}}; \text{INTERNAL } \Pi_{\text{LC}}$
FROM $\mathcal{R}_{\text{Lin}}^f$
TO $\text{EvalGate}(\mathbb{K}, C, \text{Pack})$

CURRENT STATE

LIN[f] **RELATION** **USED**
 $\langle \mathbf{a}_f, \mathbf{f}_{\mathbb{K}} \rangle_{\mathbb{K}} = \eta$

WHERE

$\rho \ \rho = \text{Root}(O)$

P7/V7. Ring-switch each active linear claim

When the coefficient LinCheck is skipped, split $\mathbf{r}_h^{(\ell)} = (r_{lo}^{(\ell)}, r_{hi}^{(\ell)})$ with $|r_{lo}^{(\ell)}| = 7$. When the coefficient LinCheck is required, use its point r instead. In both cases define

$$z_\ell := \sum_{v=0}^{127} \text{eq}(v, r_{lo}^{(\ell)}) s_v^{(\ell)}, \quad \begin{cases} z_\ell = \eta_\ell, & \text{MLE-shaped coefficient,} \\ \sigma_{m_f-1} = \tilde{a}_f(r) z_0, & \text{coefficient LinCheck.} \end{cases} \quad (47)$$

The applicable gate in Equation (47) is checked after the NARG frame is absorbed and before tag 4101; no division occurs. After the shared challenge r'' , the successor coefficients are

$$\begin{aligned} \bar{s}_u^{(\ell)} &:= \sum_v [s_v^{(\ell)}]_u \beta_v, & \Phi_{r''}(z) &:= \sum_u \text{eq}(u, r'') [z]_u, \\ \beta_\ell &:= \sum_u \text{eq}(u, r'') \bar{s}_u^{(\ell)}, & B_\ell(Y) &:= \Phi_{r''}(\text{eq}(r_{hi}^{(\ell)}, Y)). \end{aligned} \quad (48)$$

Prover P	Traffic	Verifier V
For every $\ell \in \mathcal{A}$ and $0 \leq v < 128$, compute $s_v^{(\ell)} := F_v(r_{hi}^{(\ell)})$.	$\xrightarrow{N[s^{(\ell)}]}$	For each ℓ , validate and absorb the canonical $s^{(\ell)}$ frame, then require the applicable equality in Equation (47) before any squeeze. After the last vector squeeze $r'' \leftarrow \text{Squeeze}_{\mathbb{K}}^T(\text{tr})$.
With $[z]_u$ denoting coordinate u in the fixed $\mathbb{K}/\mathbb{F}_2$ basis, derive (B_ℓ, β_ℓ) by Equation (48).	$\xleftarrow{D[r'']}$	Independently derive the same $(B_\ell, \beta_\ell)_{\ell \in \mathcal{A}}$.

P8/V8. Batch the packed linear relations

Prover P	Traffic	Verifier V
Form $B := \sum_{\ell \in \mathcal{A}} \eta_\ell B_\ell$, $\beta := \sum_{\ell \in \mathcal{A}} \eta_\ell \beta_\ell$.	$\xleftarrow{D[\eta]}$	For each active ℓ in increasing order, squeeze $\eta_\ell \in \mathbb{K}$ under tag 4102 and scope $(\ell, \perp_{32}, \perp_{32})$. Independently derive the same (B, β) .

P9/V9. Open the batched claim

Prover P	Traffic	Verifier V
Run the fixed polynomial opening protocol—Ligerito in this proof system—for the batched claim (ρ, B, β) , producing π_{open} .	$\xrightarrow{N/H[\pi_{\text{open}}]}$	Accept only if π_{open} verifies against (ρ, B, β) . This authenticates $O = \text{Enc}_C(P)$ and discharges the batched claim.

Recursive-opening channel ledger. For the recursive Ligerito protocol, the following partition is normative at the field level. The NARG records are replayed in verifier-event order, not Rust structure-field order; hint records are consumed at the opening level where their checks appear.

Class	Fields	Binding and verification deadline
$N[\cdot]$	Initial and recursive roots; sum-check messages; OOD values; <code>final_proof.yr</code> ; query- and fold-grinding nonces.	Each is observed or checked-and-absorbed before a dependent challenge. PoW nonces remain NARG because multiple values may satisfy the predicate; Section 6.7 fixes their tags and schedule.
$H[\text{open}_0]$	<code>initial_proof.opened_rows</code> and <code>initial_proof.merkle_proof</code> .	The prior initial root and sampled query positions bind the rows and path. Merkle verification must succeed before the next sumcheck-introduction challenge.
$H[\text{open}_i]$	Each recursive proof's <code>opened_rows</code> and <code>merkle_proof</code> .	Verify against the already absorbed preceding-level root and sampled queries before the next batching challenge. Consume levels in increasing recursive order.
$H[\text{open}_{\text{fin}}]$	<code>final_proof.opened_rows</code> and <code>final_proof.merkle_proof</code> .	Verify against the preceding root after the final queries are fixed and before the final batching challenge or acceptance.

The authenticated rows are hints because the earlier Merkle root computationally fixes their accepted contents; the authentication nodes only certify that fixed opening. The whole π_{open} MUST NOT be placed on the hint channel. A recursive-opening implementation must reject before any later squeeze on a failed row-length or Merkle check and must exhaust both substreams before returning success. Section 6.8 fixes the byte payload, scope, cardinality, and exact replay position of every field summarized in this table.

The Rust type `FlockCommitHint` is unrelated terminology: it is a prover-local commitment cache/key, not an H_{F2Z} record. Its rows, packed columns, message, and prover data are not proof bytes; only the public commitment/root enters the verifier-facing protocol. Steps P6/V6 through P9/V9 are the fixed recursive-opening protocol's internal expansion of the paper edge Π_{iopp} . The verifier accepts iff every displayed check succeeds and the proof parser consumes exactly the fields specified by these steps.

REDUCTION LEDGER

Π_{iopp} INTERNAL $\Pi_{\text{ring}}^{\mathcal{A}}$
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{f,(\ell)}$
TO $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{id})$

CURRENT STATE

LIN[id]^A **RELATION** **NEW**
 $\langle B_\ell, P \rangle_{\mathbb{K}} = \beta_\ell$
 $(\ell \in \mathcal{A})$

WHERE

ρ $\rho = \text{Root}(O)$
 P $P = \text{Pack}(F)$

REDUCTION LEDGER

Π_{iopp} INTERNAL Batch_η
FROM $\bigwedge_{\ell \in \mathcal{A}} \mathcal{R}_{\text{Lin}}^{(\ell)}(\mathbb{K}, C, \text{id})$
TO $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{id})$

CURRENT STATE

LIN[id] **RELATION** **NEW**
 $\langle B, P \rangle_{\mathbb{K}} = \beta$

WHERE

ρ $\rho = \text{Root}(O)$
 P $P = \text{Pack}(F)$

REDUCTION LEDGER

Π_{iopp} INTERNAL Π_{open}
FROM $\mathcal{R}_{\text{Lin}}(\mathbb{K}, C, \text{id})$
TO $\mathcal{R}_{\text{triv}}$

CURRENT STATE

nothing left to prove

5 Verifier algorithm

The verifier rejects on a failed gate, malformed value, unsupported public configuration, or cursor mismatch. NARG and hint cursors are independent. A hint remains pending until its predicate succeeds, and every squeeze requires `NoPendingHint`.

[N] read / bind	[H] read / check	[S] challenge	[C] reject gate	[D] local value	[CALL] subprotocol
P1/V1 statement	P2/V2–P3/V3 active-witness core	P4/V4–P5/V5 reconstruction / adjoint	P6/V6–P9/V9 opening		

Procedure VerifyF2Z.

Require: $\text{pp}, \mathcal{I}_{\text{LinBitsDual}} = (a, a', \mu, M), \text{com} = (\rho), \pi_{\text{host}}$
Ensure: true or false

P1/V1: Validate and bind the F2Z statement

$\mathcal{R}_{\text{LinBitsDual}}[h]$
1: $(N_{\text{F2Z}}, H_{\text{F2Z}}, n_N, n_H, b_N, b_H) \leftarrow \text{DecodeHostV1}(\pi_{\text{host}})$
2: **[C] Require**[HeaderV1 = 1 $\wedge$ wire_version = 1]
3: **[C] Require**[ValidateConfig(pp, $\mathcal{I}_{\text{LinBitsDual}}, \rho$) = 1]
4: Read $(n_f, m_f, n_h, m_h, d_r, d_c, c_w, L, g)$ from the fixed public configuration
5: **[C] Require**[$n_f = 2^{n_h} \wedge n_h = 2^{m_h} \wedge m_h = d_r + d_c \wedge c_w = 127 - d_r \wedge c_w > 0$]
6: **[C] Require**[$M \in \mathbb{F}_2^{n_h \times n_f} \wedge \text{ord}(g) = 2^{128} - 1$]
7: **[C] Require**[$|a| = 2^{d_r} \wedge |a'| = 2^{d_c} \wedge \mu \in \mathbb{F}_q$]
8: $\text{tr} \leftarrow$ **[N] Absorb**($\text{tr}_{\text{init}}, \text{Frame}_{v1}(0x0001, \perp_{32}, \perp_{32}, \perp_{32}; \text{DomainV1})$) using Equation (59)
9: $\text{tr} \leftarrow$ **[N] Absorb**($\text{tr}, \text{Frame}_{v1}(0x0002, \perp_{32}, \perp_{32}, \perp_{32}; \text{StatementV1})$) using Equation (59)
10: Initialize independent cursors $[D] c_N \leftarrow \text{Cursor}(N_{\text{F2Z}}, n_N, b_N)$, $c_H \leftarrow \text{Cursor}(H_{\text{F2Z}}, n_H, b_H)$, and $\text{PendingHint} \leftarrow 0$
11: Load the fixed GKR, row-LinCheck, and opening ledgers
12: For every row r , derive the unique $\gamma_r \in [0, q)$ with $\pi_q(\gamma_r) = a_r$
13: $[D] \gamma_r^{(\ell)} \leftarrow \lfloor \gamma_r / 2^{c_w \ell} \rfloor \bmod 2^{c_w}$
14: $[D] \mathcal{A} \leftarrow \{\ell < L : \exists r, \gamma_r^{(\ell)} \neq 0\}$; **[C] Require**[$\mathcal{A} \neq \emptyset$]
15: $\text{tr} \leftarrow$ **[N] Absorb**($\text{tr}, \text{Frame}_{v1}(0x0003, \perp_{32}, \perp_{32}, \perp_{32}; \text{CoreSeamV1})$) using Equation (60)
16: Set $\mu^{(\ell)} \leftarrow 0^{2^{d_c}}$ for $\ell \notin \mathcal{A}$
17: Fix, without advancing either cursor, the remaining record schedule
18: **for** $\ell \in \mathcal{A}$ in increasing order **do**

P2/V2: Verify chunk ℓ of Π_1

$\mathcal{R}_{\text{LinBitsDual}}[h] \xRightarrow{\Pi_1^{(\ell)}} \mathcal{R}_{\text{PESAT}}^{(\ell)}[h]$
19: $(\nu^{(\ell)}, F_\nu) \leftarrow$ **[N] ReadNarg**($c_N, 0x2001, \ell, \perp_{32}, \perp_{32}$)
20: $\text{tr} \leftarrow$ **[N] Absorb**(tr, F_ν)
21: $\text{PendingHint} \leftarrow 1$, $(\mu^{(\ell)}, F_\mu) \leftarrow$ **[H] ReadHint**($c_H, 0x2081, \ell, \perp_{32}, \perp_{32}$)
22: **[C] Require**[$|\mu^{(\ell)}| = 2^{d_c} \wedge 0 \leq \mu_c^{(\ell)} < 2^{127} \wedge (g^{\mu_c^{(\ell)}})_c = \nu^{(\ell)}$]
23: $\text{PendingHint} \leftarrow 0$
24: $\zeta^{(\ell)} \leftarrow$ **[S] Squeeze** $_{\mathbb{K}, 0x2101}^{d_c}(\text{tr}; \ell)$, $e_0^{(\ell)} \leftarrow \tilde{\nu}^{(\ell)}(\zeta^{(\ell)})$

P3/V3: Verify GKR and the sum-to-MLE reduction

$\mathcal{R}_{\text{PESAT}}^{(\ell)}[h] \xRightarrow{\Pi_{\text{GKR}}^{(\ell)}} \mathcal{R}_{\text{sum}}^{h, (\ell)} \xRightarrow{\Pi_{\text{MLE}}^{(\ell)}} \mathcal{R}_{\text{eval}}^{h, (\ell)}$
25: $[D] \mathcal{I}_{\text{GKR}}^{(\ell)} \leftarrow \text{GKRInput}(\gamma^{(\ell)}, \nu^{(\ell)}, \zeta^{(\ell)}, e_0^{(\ell)})$
26: $(\text{ok}_{\text{GKR}}, \text{tr}, \mathbf{r}_h^{(\ell)}, \eta_\ell) \leftarrow$ **[CALL] GKRAndRowLinCheck.VerifyStream**($\text{pp}_{\text{GKR}}, \mathcal{I}_{\text{GKR}}^{(\ell)}; \text{tr}, c_N, c_H$)
27: **[C] Require**[$\text{ok}_{\text{GKR}} = 1$]
28: **end for**

P4/V4: Complete the fold reconstruction

29: $[D] \mu_c^{\text{col}} \leftarrow \sum_{\ell=0}^{L-1} 2^{c_w \ell} \mu_c^{(\ell)}$ over $\mathbb{Z}$
30: **[C] Require**[$\sum_c a_c \pi_q(\mu_c^{\text{col}}) = \mu$]

P5/V5: Pull each active claim back through $M^\top$

31: **for** $\ell \in \mathcal{A}$ in increasing order **do**
32: $[D] a_h^{(\ell)}[z] \leftarrow \text{eq}(z, \mathbf{r}_h^{(\ell)})$
33: $[D] a_f^{(\ell)} \leftarrow M_{\mathbb{K}}^\top a_h^{(\ell)}$
34: **[C] Require**[$\text{FullAdjointEvaluator}(M, a_h^{(\ell)}, a_f^{(\ell)}) = 1$]
35: **end for**

P6/V6: Coefficient LinCheck when the pulled-back coefficient requires it

36: $[D] b_{\text{LC}} \leftarrow \text{NeedsLinCheck}((a_f^{(\ell)})_{\ell \in \mathcal{A}})$
37: **if** b_{LC} **then**
38: $(\text{tr}, (\mathbf{r}_f^{(\ell)}, \eta_\ell)_{\ell \in \mathcal{A}}) \leftarrow$ **[CALL] CoefficientLinCheck.VerifyStream**($\text{pp}, (a_f^{(\ell)}, \eta_\ell)_{\ell \in \mathcal{A}}; \text{tr}, c_N$)
39: The helper alternates tag-4002 messages with tag-4103 challenges and leaves the terminal gate for P7/V7
40: **else**

41: **[C] Require**[$\pi_{\text{LC}} = () \wedge \varepsilon_{\text{LC}} = 0$]

42: $[D] \mathbf{r}_f^{(\ell)} \leftarrow \mathbf{r}_h^{(\ell)}$

43: **end if**

P7/V7: Verify the ring switch

44: **for** $\ell \in \mathcal{A}$ in increasing order **do**
45: Write $\mathbf{r}_f^{(\ell)} = (r_{\text{lo}}^{(\ell)}, r_{\text{hi}}^{(\ell)})$
46: $(s^{(\ell)}, F_s) \leftarrow$ **[N] ReadNarg**($c_N, 0x4001, \ell, \perp_{32}, \perp_{32}$)
47: **[C] Require**[$|r_{\text{lo}}^{(\ell)}| = 7 \wedge |s^{(\ell)}| = 128$]
48: $\text{tr} \leftarrow$ **[N] Absorb**(tr, F_s)
49: $[D] z_\ell \leftarrow \sum_{v=0}^{127} \text{eq}(v, r_{\text{lo}}^{(\ell)}) s_v^{(\ell)}$
50: Check $z_\ell = \eta_\ell$ when $b_{\text{LC}} = 0$; otherwise check the coefficient-LinCheck terminal equation (46)
51: **end for**
52: $r'' \leftarrow$ **[S] Squeeze** $_{\mathbb{K}, 0x4101}^7(\text{tr})$
53: **for** $\ell \in \mathcal{A}$ in increasing order **do**
54: $[D] \tilde{s}_u^{(\ell)} \leftarrow \sum_v [s_v^{(\ell)}]_u \beta_v$, $\Phi_{r''}(z) \leftarrow \sum_u \text{eq}(u, r'') [z]_u$
55: $[D] \beta_\ell \leftarrow \sum_u \text{eq}(u, r'') \tilde{s}_u^{(\ell)}$, $B_\ell(Y) \leftarrow \Phi_{r''}(\text{eq}(r_{\text{hi}}^{(\ell)}, Y))$
56: **end for**

P8/V8: Batch the packed LIN claims

```

57: for  $\ell \in \mathcal{A}$  in increasing order do
58:    $\eta_{\text{bat}, \ell} \leftarrow [\text{S}] \text{Squeeze}_{\mathbb{K}, 0x4102}(\text{tr}; \ell, \perp_{32}, \perp_{32})$ 
59: end for
60:  $[D] B \leftarrow \sum_{\ell \in \mathcal{A}} \eta_{\text{bat}, \ell} B_{\ell}, \beta \leftarrow \sum_{\ell \in \mathcal{A}} \eta_{\text{bat}, \ell} \beta_{\ell}$ 

```

P9/V9: Open the batched claim

```

61: [C] Require  $[B = \text{DerivedCoefficient}(\mathcal{I}_{\text{LinBitsDual}}, \text{tr})]$ 
62:  $\text{tr} \leftarrow [\text{N}] \text{Absorb}(\text{tr}, \text{Frame}_{v1}(0x5001, \perp_{32}, \perp_{32}, \perp_{32}; \text{LE}_{32}(m_f - 7) \parallel \text{Enc}_{\mathbb{K}}(\beta)))$ 
63: [C] Require  $[[\text{CALL}] \text{RecVerifyStream}(\text{pp}, \rho, B, \beta; \text{tr}, c_N, c_H) = 1]$ 
64: [C] Require  $[\text{NoPendingHint} = 1 \wedge \text{NargEOF}(c_N) = 1 \wedge \text{HintEOF}(c_H) = 1 \wedge \text{HostEOF}(\pi_{\text{host}}) = 1 \wedge \text{Counts}(c_N, c_H) = (n_N, n_H)]$ 
65: return true

```

The helper verifiers own their subordinate ledgers. Every NARG read advances only c_N , every hint read advances only c_H , and no helper returns with an unchecked hint. P2/V2's $\text{N}[\nu^{(\ell)}]$, $\text{H}[\mu^{(\ell)}]$ order still uses tags 2001 and 2081. The byte rules in Section 6 govern every read, absorb, squeeze, PoW check, and EOF condition.

6 Wire and transcript format v1

This section fixes the bytes consumed by a conforming prover and verifier. The proof system has two physical proof streams, prefix-free typed records, a BLAKE3 transcript, SHA-256 Merkle roots, and the recursive Ligerito and proof-of-work parameters below. A proof using the legacy positional/bincode codec is not a v1 proof.

6.1 Version and fixed algorithms

The eight-byte container magic is the ASCII string F2ZPCS\0\0, and the wire version is one. Changing any normative rule in this section requires a new wire version.

The transcript and its XOF are unkeyed BLAKE3; Merkle nodes and roots use SHA-256; PoW uses unkeyed BLAKE3-256. The field basis is the polynomial basis of Equation (4). The public parameters fix the GKR protocol, constrained-code construction, transcript, and opening protocol. None is selected or negotiated by proof bytes. A proof system without a byte-exact GKR ledger or without a canonical codeword construction and root vectors is incomplete and MUST NOT accept proofs.

Write $m := m_f$ and $m_p := m - 7$ for the committed layout. For the deployed range $22 \leq m \leq 35$, let J_m be the number of recursive opening levels. The level shapes and all parameters except the base fold difficulties are fixed by

$$\begin{aligned}
 h_i &:= m - 11 - 3i, & J_m &:= \min\{J \geq 1 : h_{J-1} \in \{3, 4, 5\}\}, \\
 r_i &:= 1 + i, & k_i &:= \begin{cases} 4, & i = 0, \\ 3, & i > 0, \end{cases} \\
 Q_i &:= (183, 90, 60, 45, 36, 31, 27, 24)_i, & g_i &:= 16, \\
 o_i &:= \begin{cases} 0, & i = 0, \\ 1, & i > 0. \end{cases}
 \end{aligned} \tag{49}$$

Here h_i is the message-column exponent, r_i the inverse-rate exponent, k_i the fold count, Q_i the query count, g_i the query-grinding difficulty, and o_i the OOD sample count. Implementations MUST use the integers in Equation (49), not a floating-point recomputation.

The remaining base fold difficulties f_i are the following literal table. Only the first J_m entries of the row for m exist.

m	$(f_0, \dots, f_{J_m-1})$
22	(9, 6, 3)
23	(10, 7, 4, 1)
24	(11, 8, 5, 2)
25	(12, 9, 6, 3)
26	(13, 10, 7, 4, 2)
27	(14, 11, 8, 5, 3)
28	(15, 12, 9, 6, 4)
29	(16, 13, 10, 7, 5, 3)
30	(17, 14, 11, 8, 6, 4)
31	(18, 15, 12, 9, 7, 5)
32	(19, 16, 13, 10, 8, 6, 3)
33	(20, 17, 14, 11, 9, 7, 4)
34	(21, 18, 15, 12, 10, 8, 5)
35	(22, 19, 16, 13, 11, 9, 6, 6)

6.2 Primitive canonical encodings

All unsigned integers are fixed-width little-endian. Decoders MUST NOT reduce, truncate, ignore high bits, or convert to a host-sized integer before checking that the value fits. The following encodings are the only canonical ones.

Type	Bytes	Canonicity rule
$u_8, u_{16}, u_{32}, u_{64}, u_{128}$	1, 2, 4, 8, 16	Unsigned little-endian of exactly the named width.
$a \in \mathbb{K}$	16	$\text{LE}_{64}(a_{\text{lo}}) \parallel \text{LE}_{64}(a_{\text{hi}})$ in the fixed polynomial basis; coefficient bit j is bit j of the resulting 128-bit word.
$a \in \mathbb{F}_q$	16	$\text{LE}_{128}(\bar{a})$, where $0 \leq \bar{a} < q$ and $\pi_q(\bar{a}) = a$; reject an integer at least $q = 2^{100} - 15$.
Chunk exponent u	16	$\text{LE}_{128}(u)$; require $u < 2^{127}$.
Nonce	8	$\text{LE}_{64}(n)$.
Merkle value	32	Raw SHA-256 digest bytes, in digest output order.
Vector $[x_0, \dots, x_{n-1}]$	4 plus elements	$\text{LE}_{32}(n)$ followed by canonical elements in index order; n must equal the protocol-derived expected count.
Opened rows and path	nested	Vector of rows, each a vector of $\mathbb{K}$ elements, followed by a vector of 32-byte Merkle values. Expected row, width, and path counts are derived before allocation.

Every composite is concatenation without padding. Options, Rust enums, **usize**, variants, serde defaults, and bincode layouts are not wire types. For every accepted primitive or composite x , decoding its encoding must consume the whole bounded slice and re-encoding the result must reproduce the same bytes.

6.3 Merkle rows and canonical multiproofs

For opening level $0 \leq k < J_m$, define the message dimension, tree depth, leaf count, and row width by

$$n_k := h_k + k_k, \quad D_k := h_k + r_k, \quad \Lambda_k := 2^{D_k}, \quad I_k := 2^{k_k}. \tag{50}$$

Equation (50) implies $n_0 = m - 7$, $n_k = h_{k-1}$ for $k > 0$, $D_k = m - 10 - 2k$, $I_0 = 16$, and $I_k = 8$ for $k > 0$. Let $C_k \in \mathbb{K}^{\Lambda_k \times I_k}$ be the level- k codeword matrix, with leaves and lanes in increasing integer order. Its exact row and tree bytes are

$$\begin{aligned} \text{RowBytes}_k(q) &:= \text{Enc}_{\mathbb{K}}(C_k[q, 0]) \parallel \dots \parallel \text{Enc}_{\mathbb{K}}(C_k[q, I_k - 1]), \\ H_{k,0,q} &:= \text{SHA256}(\text{RowBytes}_k(q)), \\ H_{k,d+1,j} &:= \text{SHA256}(H_{k,d,2j} \parallel H_{k,d,2j+1}), \\ \text{root}_k &:= H_{k,D_k,0}. \end{aligned} \quad (51)$$

Here $\text{Enc}_{\mathbb{K}}$ is the 16-byte encoding in Section 6.2. Equation (51) uses no native-memory cast, row-length prefix, row index, level tag, padding, or leaf/internal-node domain byte. Adding such a byte changes every root and requires a new wire version. The fixed code specification defines how C_k is generated; this section fixes its portable row and tree encoding.

Let the already-derived query set be $A_0 = (q_0 < \dots < q_{Q_k-1})$. At depth d , scan the active positions A_d in increasing order. If both siblings are active, combine the pair once and emit no digest. Otherwise emit the digest at position $p \text{ xor } 1$, using p 's parity to place it on the left or right. Then set $A_{d+1} := \text{sortuniq}\{[p/2] : p \in A_d\}$. The canonical multiproof is the concatenation of emitted digests, bottom-up and in that scan order, and its exact count is

$$M_k := \sum_{d=0}^{D_k-1} |\{p \in A_d : (p \text{ xor } 1) \notin A_d\}|. \quad (52)$$

Equation (52) fixes the exact number of transmitted path digests. The level- k opening payload is exactly

$$\text{LE}_{32}(Q_k) \parallel \prod_{j=0}^{Q_k-1} (\text{LE}_{32}(I_k) \parallel \text{RowBytes}_k(q_j)) \parallel \text{LE}_{32}(M_k) \parallel \prod_{u=0}^{M_k-1} \text{path}_k[u], \quad (53)$$

with byte length

$$8 + Q_k(4 + 16I_k) + 32M_k. \quad (54)$$

The verifier derives A_d , M_k , and Equation (54) from the sampled queries before allocating or reading the bounded Hint record, requires rows in q_j order, reconstructs Equation (51), and rejects unused path digests or payload bytes.

6.4 Outer two-stream container

The 40-byte header is fixed as follows. Offsets and sizes are decimal; all multibyte integers use Section 6.2.

offset	size	field
0	8	magic = 46 32 5a 50 43 53 00 00
8	2	wire_version = 01 00
10	2	header_len = 28 00 (40)
12	4	flags = 00 00 00 00
16	4	narg_record_count
20	4	hint_record_count
24	8	narg_byte_len
32	8	hint_byte_len
40	...	NARG records, then Hint records

Writing Hdr_{40} for those exact header bytes, the only v1 host encoding is

$$\text{HostEncode}_{v1}(N, H) := \text{Hdr}_{40} \parallel N \parallel H, \quad \text{bytelen}(\text{HostEncode}_{v1}(N, H)) = 40 + \text{bytelen}(N) + \text{bytelen}(H). \quad (55)$$

Equation (55) implies the four benchmark metrics

$$\begin{aligned} \text{narglen}(\pi) &= \text{bytelen}(N), & \text{hintlen}(\pi) &= \text{bytelen}(H), \\ \text{payloadlen}(\pi) &= \text{bytelen}(N) + \text{bytelen}(H), & \text{wirelen}(\pi) &= 40 + \text{payloadlen}(\pi). \end{aligned} \quad (56)$$

The decoder MUST verify Equation (56) with checked arithmetic before allocating, enforce statement-derived upper bounds, and require input length exactly $40 + \text{narg_byte_len} + \text{hint_byte_len}$. The two declared record counts must match the records actually consumed. Header count and length fields are transport metadata and are not transcript input.

6.5 Typed event frames and registry

Every NARG record, Hint record, public absorb, and squeeze event uses the same prefix-free frame header:

offset	size	field
0	2	event_tag u16-le
2	2	reserved = 0 u16-le
4	4	chunk u32-le
8	4	level u32-le
12	4	round_or_index u32-le
16	8	payload_len u64-le
24	...	payload[payload_len]

Let $\perp_{32} := \text{0xffffffff}$ denote an inapplicable scope coordinate. For tag τ , scope (ℓ, k, i) , and payload z , the frame is exactly

$$\text{Frame}_{v1}(\tau, \ell, k, i; z) := \text{LE}_{16}(\tau) \parallel \text{LE}_{16}(0) \parallel \text{LE}_{32}(\ell) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{64}(\text{bytelen}(z)) \parallel z. \quad (57)$$

Thus Equation (57) has length $24 + \text{bytelen}(z)$. A stream record with a reserved nonzero word, an inapplicable coordinate other than $\perp_{32}$, a wrong channel, a wrong expected scope, or an unknown tag is malformed.

The base-tag registry is below. D events are public or derived, have no proof record, and are absorbed at their ledger position; N records occur only in N and are absorbed; H records occur only in H and are checked but never absorbed; S tags are squeeze requests. For an S base tag τ , the squeeze-result tag is $\tau \text{ OR } 0x8000$ and is derived, never transmitted. For an S row, the last column describes the derived result; its request payload is always $\text{LE}_{32}(n) \parallel a$ as fixed by Equation (61).

Tag	Class	Event	Scope; proof payload or squeeze result
0001	D	domain/version	$(\perp, \perp, \perp)$; DomainV1 from Equation (59).
0002	D	statement	$(\perp, \perp, \perp)$; StatementV1 from Equation (59).
0003	D	core seam	$(\perp, \perp, \perp)$; CoreSeamV1 from Equation (60).
1000-1fff	—	reserved	These tags MUST NOT occur in an F2Z proof.
2001	N	chunk ν	$(\ell, \perp, \perp)$; vector of 2^{d_c} elements of $\mathbb{K}$.
2081	H	chunk μ	$(\ell, \perp, \perp)$; vector of 2^{d_c} canonical $u_{128} < 2^{127}$.
2101	S	chunk ζ	$(\ell, \perp, i)$ for $0 \leq i < d_c$; one $\mathbb{K}$ challenge.
3001	N	GKR phase-c sum	$(\ell, k, \perp)$; one $\mathbb{K}$ element, absorbed after the phase-link check and before the first phase-c round.
3002-30ff	GKR	GKR messages	The fixed GKR protocol owns exact message tags, codecs, and checks in this range.
3100-31ff	GKR	GKR squeezes	The fixed GKR protocol owns exact squeeze tags and result types in this range.
4001	N	ring s vector	$(\ell, \perp, \perp)$; vector of exactly 128 elements of $\mathbb{K}$.

Tag	Class	Event	Scope; proof payload or squeeze result
4002	N	coefficient-LinCheck round	$(\perp, \perp, i)$ for $0 \leq i < m_f$; exactly the three coefficients of $g_i(X) = \gamma_{i,0} + \gamma_{i,1}X + \gamma_{i,2}X^2$, encoded as $\text{Enc}_{\mathbb{K}}(\gamma_{i,0}) \parallel \text{Enc}_{\mathbb{K}}(\gamma_{i,1}) \parallel \text{Enc}_{\mathbb{K}}(\gamma_{i,2})$, with no vector prefix.
4101	S	shared r''	$(\perp, \perp, i)$ for $0 \leq i < 7$; one $\mathbb{K}$ challenge.
4102	S	batch η	$(\ell, \perp, \perp)$ in increasing active-chunk order; one $\mathbb{K}$ challenge.
4103	S	coefficient-LinCheck challenge	$(\perp, \perp, i)$ for $0 \leq i < m_f$; one $\mathbb{K}$ challenge.
5001	D	opening target	$(\perp, \perp, \perp)$; $\text{LE}_{32}(m_f - 7) \parallel \text{Enc}_{\mathbb{K}}(\beta)$. B is recomputed.
5002	N	initial root	$(\perp, 0, \perp)$; one 32-byte SHA-256 digest.
5003	N	recursive root	$(\perp, k, \perp)$, $1 \leq k < J_m$; one digest.
5004	N	sumcheck message	scope by kind below; kind: u_8 followed by $\text{Enc}_{\mathbb{K}}(u_0) \parallel \text{Enc}_{\mathbb{K}}(u_2)$.
5005	N	OOD value	$(a, k, \perp)$; one $\mathbb{K}$ element for sample a .
5006	N	final y_r	$(\perp, J_m - 1, \perp)$; vector of exactly $2^h J_m - 1$ elements of $\mathbb{K}$.
5007	N	fold-PoW nonce	$(\perp, k, i)$; one u_{64} .
5008	N	query-PoW nonce	$(\perp, k, \perp)$; one u_{64} .
5080	H	initial opening	$(\perp, 0, \perp)$; Equation (53).
5081	H	recursive opening	$(\perp, k, \perp)$, $1 \leq k < J_m - 1$; Equation (53).
5082	H	final opening	$(\perp, J_m - 1, \perp)$; Equation (53).
5101	S	fold challenge	$(\perp, k, i)$; one $\mathbb{K}$ challenge.
5102	S	OOD point	(a, k, c) ; one $\mathbb{K}$ coordinate, with c increasing.
5103	S	glue challenge	$(\perp, k, i)$ as fixed below; one $\mathbb{K}$ challenge.
5104	S	PoW seed	$(\perp, k, i)$; 16 bytes; request auxiliary payload C .
5105	S	query draw	$(\perp, k, a)$; one $\mathbb{K}$ challenge for attempt a .
5106	S	query alpha	$(\perp, k, c)$; one $\mathbb{K}$ challenge.

The GKR ranges are black-box namespaces, not permission to use implementation-defined bytes. The fixed GKR protocol must publish a ledger with the same columns as this table. The fixed 3001 event resolves the PCS-level phase- c placement ambiguity in Gap G1; the rest of that gap remains GKR-protocol work and no protocol may reassign the tag. The v1 outer row LinCheck omits the redundant `presums[ell].claimed_sums` field and derives $[e_d^{(\ell)} + 1]$, so tag 3081 is reserved and MUST NOT occur.

6.6 Transcript transitions and replay

Model the transcript state as the byte string hashed by an unkeyed BLAKE3 hasher. It starts empty. The domain/version frame 0001 and statement frame 0002 are absorbed, in that order, before any proof-dependent squeeze. The application-supplied linear-claim statement digest is

$$d_{\text{claim}} := \text{BLAKE3}_{256}(\text{hex}^{-1}(46325a2d5043532f6170702d73746174656d656e742f763100) \parallel \text{linear_claim_statement_bytes}). \quad (58)$$

For v1 byte compatibility, Equation (58) retains the legacy ASCII domain literal F2Z-PCS/app-statement/v1 followed by one NUL byte. Changing that literal requires a new wire version. The two initial frame payloads are exactly

$$\begin{aligned} \text{DomainV1} &:= \text{hex}^{-1}(46325a5043530000) \parallel \text{LE}_{16}(1), \\ \text{StatementV1} &:= d_{\text{claim}}[32] \parallel \text{LE}_{32}(d_r) \parallel \text{LE}_{32}(d_c) \parallel \text{LE}_{32}(m_f) \\ &\quad \parallel \text{Enc}_{\mathbb{K}}(g) \parallel \rho[32]. \end{aligned} \quad (59)$$

Thus DomainV1 and StatementV1 have 10 and 92 bytes, respectively. The host application must make the linear-claim statement byte string in Equation (58) canonical.

The derived core-seam payload is

$$\text{CoreSeamV1} := \text{LE}_{32}(2^{d_r}) \parallel \prod_{r=0}^{2^{d_r}-1} \text{LE}_{128}(\gamma_r) \parallel \text{LE}_{32}(2^{d_c}) \parallel \prod_{c=0}^{2^{d_c}-1} \text{LE}_{128}(\tilde{a}'_c) \parallel \text{LE}_{128}(\tilde{\mu}), \quad (60)$$

where $\pi_q(\tilde{a}'_c) = a'_c$, $0 \leq \tilde{a}'_c < q$, and $\pi_q(\tilde{\mu}) = \mu$, $0 \leq \tilde{\mu} < q$. It has exactly $24 + 16(2^{d_r} + 2^{d_c})$ bytes. It is absorbed under tag 0003 after the F2Z statement is bound and before the first tag-2001 record.

For a canonical frame F , absorption appends the exact frame bytes. A squeeze under base tag $\tau < 0x8000$, scope (ℓ, k, i) , output length n , and auxiliary request payload a is the transition

$$\begin{aligned} R &:= \text{Frame}_{v1}(\tau, \ell, k, i; \text{LE}_{32}(n) \parallel a), \\ \text{tr}' &:= \text{tr} \parallel R, \\ z &:= \text{BLAKE3-XOF}(\text{tr}')[0 \dots n], \\ \text{tr}'' &:= \text{tr}' \parallel \text{Frame}_{v1}(\tau \text{ OR } 0x8000, \ell, k, i; z). \end{aligned} \quad (61)$$

Equation (61) uses a clone/finalize-XOF of the incremental hasher after R ; it does not reset or key the hasher. A $\mathbb{K}$ challenge has $n = 16$ and decodes by Section 6.2. A vector challenge is a sequence of scalar squeezes, not one long squeeze. Unless its registry row says otherwise, the scalars use consecutive i values and the auxiliary request payload a is empty. Tag 4102 instead uses $(\ell, \perp, \perp)$ for each active ℓ in increasing order. Tag 5103 uses its one-byte glue kind, and tag 5104 uses the PoW context C ; these are the only nonempty core auxiliary payloads.

On a normal NARG read, the verifier validates the expected tag, scope, payload length, and canonical payload, then immediately appends the complete frame from Equation (57). A PoW nonce is the sole deferred NARG read: the verifier parses its expected canonical frame, permits no intervening event, checks Equation (63), and appends the frame only on success. On a Hint read, it validates the same framing but never appends those bytes. It must complete the hint predicate before the next squeeze. In particular, it reads and absorbs 2001, then reads 2081 and verifies the exponent bounds and $g^\mu = \nu$ before the first 2101 event. NARG replay follows verifier-event order, never Rust structure-field order. Acceptance requires no deferred record or pending hint and exact logical and byte exhaustion of both streams and every nested slice.

Tags 4002 and 4103 are enabled only when the pulled-back coefficient requires the coefficient protocol LinCheck_f . An MLE-shaped coefficient requires zero such events. Otherwise the verifier alternates, for $i = 0, \dots, m_f - 1$, one tag-4002 read-and-absorb, the check $g_i(0) + g_i(1) = \sigma_{i-1}$, and one tag-4103 squeeze at the same scope. No event may intervene within a round. It then reads and absorbs the singleton tag-4001 vector at scope $(0, \perp, \perp)$, derives $z = \sum_v \text{eq}(v, r_0)s_v$, and checks $\sigma_{m_f-1} = \tilde{a}_f(r)z$ before tag 4101. The coefficient evaluator is verifier-derived and contributes no payload. This is the only coefficient LinCheck in the opening path; the same tag-4001 vector continues into the inherited ring-switch computation.

6.7 Grinding and query derivation

Let kind byte zero denote a fold grind and kind byte one a query grind. For kind K_{pow} , level k , round i , and derived difficulty d , define the exact PoW context

$$C := \text{ASCII}(\text{F2Z-PCS/PoW/v1}) \parallel \text{LE}_8(K_{\text{pow}}) \parallel \text{LE}_{32}(k) \parallel \text{LE}_{32}(i) \parallel \text{LE}_{16}(d). \quad (62)$$

In Equation (62), $i = \perp_{32}$ for a query grind. The seed is the 16-byte output of Equation (61) under tag 5104, the same scope, and auxiliary request payload C . After obtaining the seed, the verifier reads the expected 5007 or 5008 NARG frame without absorbing it and requires, for the scheduled positive difficulty d ,

$$\text{LZ}(\text{BLAKE3}_{256}(C \parallel \text{seed}_{16} \parallel \text{LE}_{64}(n))) \geq d, \quad d > 0. \quad (63)$$

LZ in Equation (63) counts zero bits from the most-significant bit of digest byte zero, then proceeds toward the least-significant bit and subsequent bytes. Any satisfying nonce is accepted; minimality is not checked. Only after the predicate succeeds does the verifier absorb the complete nonce frame, before any dependent squeeze. Difficulty zero has no predicate invocation, seed squeeze, or nonce record in this wire format.

The exact scheduled difficulties are

$$d_k^{\text{query}} := g_k = 16, \quad d_{k,i}^{\text{fold}} := \max(f_k - i, 0). \quad (64)$$

Equation (64) is evaluated with ordinary nonnegative integers. After the preceding sumcheck message and immediately before fold challenge (k, i) , perform both the tag-5104 seed squeeze and tag-5007 nonce record if $d_{k,i}^{\text{fold}} > 0$; if the difficulty is zero, perform neither event. For $0 \leq k < J_m - 1$, query event k occurs after the tag-5003 commitment root for level $k + 1$, all o_{k+1} OOD bindings for that root, and their glue challenges, and immediately before the tag-5105 draws that open level k . Query event $J_m - 1$ instead occurs after the tag-5006 final y_r , with no new root or OOD binding, and immediately before the final-level draws. There is exactly one query nonce at every level. Consequently the exact number of nonce records is

$$N_{\text{pow}} = J_m + \sum_{k=0}^{J_m-1} \min(k_k, f_k). \quad (65)$$

Missing, extra, reordered, or trailing nonces violate Equation (65) and reject.

For level k , each 5105 event yields $r \in \mathbb{K}$ and proposes

$$q := r_{10} \bmod 2^{h_k + r_k}. \quad (66)$$

The verifier repeats Equation (66) until Q_k distinct positions exist, discarding duplicates, and then sorts the positions in increasing order. It then derives the tag-5106 alphas before reading and checking the opening.

6.8 Recursive-opening replay ledger

This subsection fixes the complete replay order for the recursive opening. The four canonical tag-5004 message kinds and scopes are

0x00	Start	$(\perp, 0, \perp)$	exactly once,	
0x01	FoldNext	$(\perp, k, j)$	$0 \leq j < k_k$,	
0x02	OodIntro	$(a, k, \perp)$	$1 \leq k < J_m, 0 \leq a < o_k$,	
0x03	QueryIntro	$(\perp, k, \perp)$	$0 \leq k < J_m - 1$.	(67)

Equation (67) is the complete tag- 5004 kind registry. Each payload is the one-byte kind followed by the two canonical $\mathbb{K}$ elements in the registry. Any other kind/scope pair rejects.

The outer VerifyF2Z caller first requires that B is the deterministic coefficient function derived in P9/V9, with domain size 2^{m_p} ; B is never densely serialized. The caller alone absorbs tag 5001 with payload $\text{LE}_{32}(m_p) \parallel \text{Enc}_{\mathbb{K}}(\beta)$ and then invokes `RecVerifyStream`. That helper starts after tag 5001: its first event reads tag 5002, requires the root to equal ρ , absorbs it, and then reads and absorbs the Start tag-5004 message.

For each $k = 0, \dots, J_m - 1$ in increasing order, it then performs the following ledger without interleaving another event.

1. For $j = 0, \dots, k_k - 1$, perform the optional fold grind exactly as in Section 6.7, squeeze tag 5101 at $(\perp, k, j)$, then read, validate, and absorb the FoldNext tag-5004 message at the same scope.
2. If $k < J_m - 1$, read and absorb the tag-5003 root at $(\perp, k + 1, \perp)$. For each $a = 0, \dots, o_{k+1} - 1$, squeeze the n_{k+1} tag-5102 coordinates at scopes $(a, k + 1, c)$ for $c = 0, \dots, n_{k+1} - 1$; read and absorb tag 5005 at $(a, k + 1, \perp)$; read and absorb the OodIntro tag-5004 message at that scope; and squeeze tag 5103 at $(\perp, k + 1, a)$ with the one-byte auxiliary kind 00.
If instead $k = J_m - 1$, read and absorb tag 5006 at $(\perp, k, \perp)$, requiring exactly 2^{h_k} canonical $\mathbb{K}$ elements.
3. Perform the query grind at $(\perp, k, \perp)$ with difficulty 16 and absorb its validated tag-5008 nonce. Starting with attempt $a = 0$, squeeze tag 5105 at $(\perp, k, a)$ and increment a after every squeeze, including a duplicate, until Q_k distinct positions have been obtained. Reject rather than wrap if the next attempt would exceed $2^{32} - 1$. Sort the positions, then squeeze exactly $\lceil \log_2 Q_k \rceil$ tag-5106 alphas at $(\perp, k, c)$ for consecutive c .
Read exactly one opening Hint using tag 5080 for $k = 0$, tag 5081 for $0 < k < J_m - 1$, or tag 5082 for $k = J_m - 1$. Check Equations (53)–(54) and the root before any later event. For $k < J_m - 1$, next read and absorb the QueryIntro tag-5004 message. At every level, including the final level where no such message exists, squeeze the query glue tag 5103 at $(\perp, k, o_k)$ with one-byte auxiliary kind 01. At the final level this challenge is used directly in the residual check.

The fixed record/event cardinalities are therefore

$$\begin{aligned}
 \#5002 &= 1, & \#5003 &= J_m - 1, & \#5005 &= \sum_{k=1}^{J_m-1} o_k, \\
 \#5004 &= 1 + \sum_{k=0}^{J_m-1} k_k + \sum_{k=1}^{J_m-1} o_k + (J_m - 1), & \#5006 &= 1, & \#508* &= J_m, \\
 \#5101 &= \sum_{k=0}^{J_m-1} k_k, & \#5102 &= \sum_{k=1}^{J_m-1} o_k n_k, & \#5103 &= J_m + \sum_{k=1}^{J_m-1} o_k, \\
 \#5106 &= \sum_{k=0}^{J_m-1} \lceil \log_2 Q_k \rceil.
 \end{aligned} \quad (68)$$

The cardinalities in Equation (68) are exact. Here #508* is one opening Hint per level. Tag-5105 count is the transcript-derived number of attempts, all of which appear in the audit log. Nonce counts remain Equation (65). A different order, scope, kind, cardinality, or unread field rejects.

6.9 Parser, implementation, and conformance requirements

A v1 implementation MUST:

1. validate magic, version, flags, all public-parameter-, statement-, and transcript-derived bounds, and all reserved fields before allocating or doing proof work;
2. use checked arithmetic and checked conversion for every length and index, reject noncanonical field values, and return a decoding error rather than panic on adversarial bytes;
3. encode the NARG stream in the exact absorb order and the Hint stream in its exact check order, with independent cursors;
4. replace opaque recursive-proof serialization with the field records and exact order in Sections 6.5 and 6.8; and
5. reject unless every outer, subordinate, recursive, row, and Merkle-path slice is exactly exhausted.

Conformance fixtures MUST include canonical primitive and frame hex, one complete $N/H/\text{HostEncode}_{v1}$ proof, the ordered transcript audit log, every squeezed challenge, PoW seeds/nonces and query positions, and all four lengths from Equation (56). Negative fixtures MUST cover an unknown version or tag, a reserved bit, wrong endian or channel, record reordering, truncation and suffix bytes, noncanonical scalars, count overflow, a failed or late u/ν hint, altered PoW scope/difficulty/nonce, and missing or extra recursive fields. A coefficient-LinCheck-enabled fixture must additionally cover a wrong round polynomial, reordered 4002/4103 pair, altered terminal coefficient, and failed terminal gate. Native, recursive, and independent reference verifiers must produce byte-identical transcript audit logs.

The repository file `wire-v1-vectors.txt` is the normative initial known-answer set for the primitive container/frame codec, Merkle row/tree/multiproof codec, and PoW predicate at difficulties 1, 7, 8, and 16. A conforming implementation must reproduce it exactly and add a complete proof/transcript fixture before claiming end-to-end v1 interoperability.